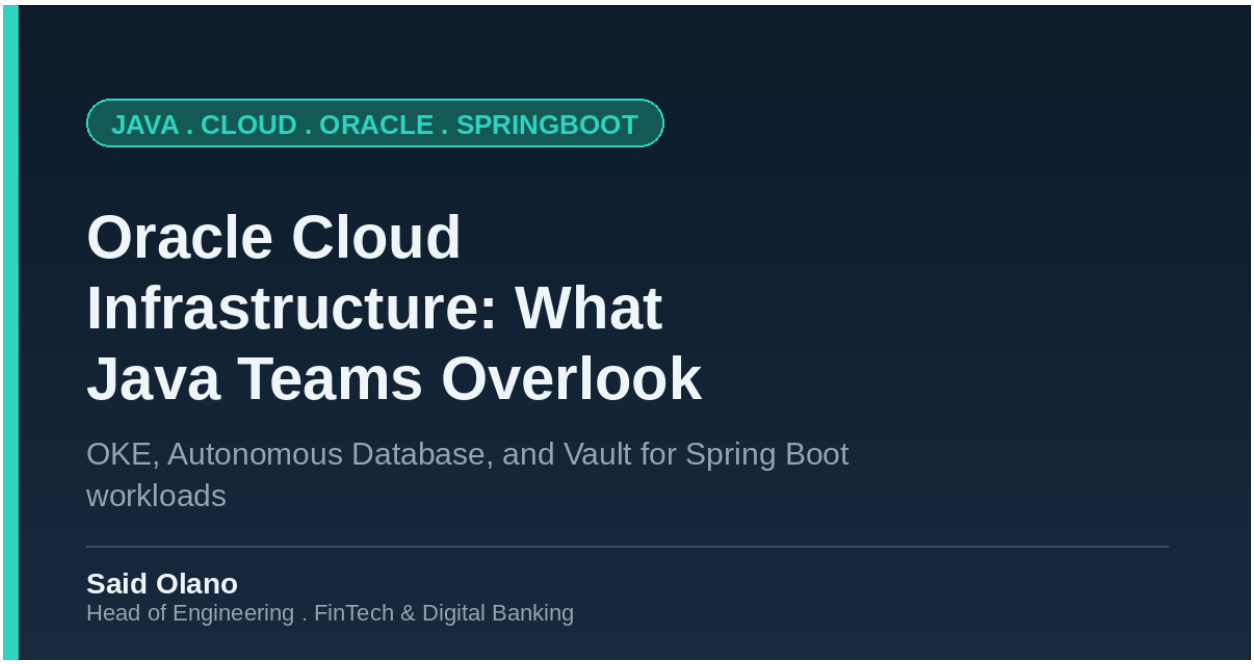

Oracle Cloud Infrastructure: What Java Teams Overlook

Most Java shops default to AWS or Azure without ever pricing out Oracle Cloud Infrastructure (OCI). That's understandable — the mindshare isn't there. But if you're running Java workloads, especially anything touching Oracle Database, OCI has a few advantages that are easy to miss until you're stuck paying egress fees or spinning up yet another managed Kubernetes cluster elsewhere.

The problem with defaulting to "everyone uses AWS"

Cloud decisions in most engineering orgs are made once, early, and rarely revisited. That's fine until the bill arrives. Two things catch teams off guard on other providers: outbound data transfer costs, and the price of running always-on compute for workloads that don't need it. Neither is unique to any one vendor, but OCI's pricing model — flat, predictable, and notably cheaper on egress — is worth a second look if you haven't priced it since your last vendor comparison.

What's actually useful for a Java stack

A few OCI services map directly onto things a typical Spring Boot / microservices team already needs:

- **Autonomous Database** — a self-patching, self-tuning Oracle DB. If your app already talks to Oracle via JDBC, this removes a meaningful chunk of DBA overhead.
- **Container Engine for Kubernetes (OKE)** — a managed Kubernetes offering that's a drop-in target for the same Helm charts and manifests you're already running elsewhere.
- **OCI Functions** — serverless, built on the open-source Fn Project, which means it's portable rather than a proprietary dead end.

• **Vault** — centralized secrets and key management, the same shape as AWS Secrets Manager or HashiCorp Vault, if you'd rather not stand up your own.

None of this requires rewriting anything. A Spring Boot service that already externalizes its datasource config can point at an Autonomous Database instance with a one-line change:

```
spring:
  datasource:
    url: jdbc:oracle:thin:@myatp_high?TNS_ADMIN=/opt/oracle/wallet
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    driver-class-name: oracle.jdbc.OracleDriver
```

The wallet-based connection handles TLS and failover for you, which is one less thing to hand-roll in a `HikariConfig`.

Where it falls short

OCI's ecosystem and community size aren't close to AWS's. Fewer Terraform modules, fewer Stack Overflow answers, fewer third-party integrations out of the box. Multi-region presence is thinner too, so if your latency requirements are global and specific, check region coverage before committing. And if your team has years of AWS-specific tooling and muscle memory, the migration cost of switching wholesale rarely pencils out just for egress savings.

Where it's worth evaluating

The case for OCI is strongest when you're already an Oracle Database shop, when your workloads are read-heavy with a lot of outbound traffic, or when you're standing up something greenfield and cost predictability matters more than ecosystem breadth. It's rarely a wholesale replacement for AWS or Azure — more often it earns a spot in a multi-cloud strategy for the specific workloads it's actually good at.

Have you evaluated OCI for a Java workload, or is AWS/Azure still the unquestioned default on your team?

Said Olano — Head of Engineering, FinTech & Digital Banking